Internal APIs Are All You Need

Shadow APIs, Shared Discovery, and the Case Against Browser-First Agent Architectures

Lewis Tham* Nicholas Mac Gregor Garcia† Jungpil Hahn‡

April 2026 (arXiv:2604.00694v1)

Abstract

Autonomous agents increasingly interact with the web, yet most websites remain designed for human browsers: an agent that wants a single fact or action loads a full page, executes its scripts, and waits on late requests to recover a call the site's own front-end already knew how to make. We observe that this cost is paid over and over for information that does not change between callers, and that the structured request behind a page — a *shadow API*, the first-party endpoint the front-end calls — is learnable once and replayable by anyone. We present Unbrowse, a system that passively learns callable routes from real usage and stores them in a shared index, so that an intent resolves to a ranked endpoint shortlist and executes against the live site without a browser. We frame the economics as a single adoption condition — a cost-minimising agent reuses a shared route if and only if the route fee undercuts the cost of rediscovering it — and settle paid execution over x402 so the indexer who mapped a route, the platform, and the site owner are all compensated in one transaction. Across 94 live domains we measure a **3.6× mean** and **5.4× median** speedup over a browser-automation baseline, with a best single-domain speedup of 30×. The short version: for the overwhelming majority of agent web tasks, internal APIs are all you need.

1 Introduction

The web was built for humans to navigate pages. Agents do not want pages — they want actions. When an autonomous agent is asked for the price of a product, the status of an order, or the results of a search, it does not need the page's markup, its analytics scripts, or its cookie banner; it needs the one structured request the page makes to its own backend, and the structured response that comes back. Browser-first agent architectures recover that request the expensive way: they render the whole page, run its JavaScript, and observe the resulting network traffic, paying a full browser's latency and token cost on every call.

We make one claim and try not to oversell it: for the overwhelming majority of agent web tasks, the browser is unnecessary overhead, because the first-party endpoint behind the page — the *shadow API* — can be learned once and replayed by any later agent. The discovery cost should be paid a single time and amortised across all callers, not re-paid on every visit. This paper presents Unbrowse, a system built on that observation, and makes three contributions:

1. A passive route-discovery pipeline that learns callable interfaces from real usage and normalises them into a shared graph. [shipped]

*Unbrowse AI, lewis@unbrowse.ai.

†School of Computing, National University of Singapore, ngarcia@nus.edu.sg.

‡School of Computing, National University of Singapore, jungpil@nus.edu.sg.

2. An adoption condition that states precisely when a shared route commons is the cost-minimising choice, and a three-tier payment architecture that settles the resulting value over x402. [shipped]

3. An evaluation across 94 live domains showing a 3.6× mean and 5.4× median speedup over a browser baseline, with the reuse mechanism isolated in a runnable reference benchmark. [reference]

2 Related work

2.1 Web agents and browser automation

A large body of work has agents drive real browsers and act on rendered pages: end-to-end multimodal web agents [3], and benchmarks such as Mind2Web [4] and WebArena [5] that measure agents on live or simulated sites. This line treats the rendered page as the interface. We take the opposite stance: the rendered page is a fallback, and the durable interface is the first-party API the page already calls.

2.2 Tool learning and API discovery

Toolformer teaches a model *when* to call an API [6] and Gorilla teaches it to emit *correct* calls across many APIs [7]. These solve single-agent tool use against *declared* interfaces. They do not address *undeclared* first-party endpoints, nor what happens when many agents rediscover the same interface repeatedly — the shared-discovery problem we take up here.

2.3 API aggregation platforms

Aggregators and standard connectors expose curated, declared APIs behind a uniform surface. Their coverage is bounded by what providers choose to publish. Shadow-API discovery is complementary and unbounded in a different direction: it learns the endpoints a site already serves to its own front-end, whether or not they are documented.

2.4 Agent economies and communication protocols

The Model Context Protocol standardises how a model connects to tools and data [8]; x402 revives HTTP 402 as a real micropayment rail [9]. Prior agent-payment work largely stops at "the agent can pay." We extend it to "the payment routes to everyone who created the value" and frame the commons in the tradition of Ostrom's work on common-pool resources [10].

3 System design

3.1 System architecture

Unbrowse resolves an intent along a three-path model: a cache-first lookup against the shared route graph, a graph traversal over typed endpoint edges, and — only on a miss — a one-time browser capture that feeds the graph for every later caller. The two-call contract an agent sees is deliberately small: an *intent* → *ranked endpoint shortlist* → *execute* resolution returns the candidates that answer an intent, and a single follow-up realizes the chosen endpoint call against the live site. [shipped]

3.2 Capability layer: route discovery pipeline

Traffic observation and filtering. During a capture session the runtime observes the requests a page makes and filters out noise (static assets, analytics, telemetry), keeping the candidate first-party data endpoints.

Route extraction and normalisation. Captured requests are normalised into routes: method, URL template with typed path and query parameters, the auth handshake, and a fingerprint of the response shape.

What the graph stores. The graph stores callable routes and typed edges (list→detail, pagination, auth), plus a signed record of which routes were used and what they returned — value off-chain, commitment hash-chained. [reference]

Skill packaging. Related routes for a domain are packaged into a reusable skill that later agents install and execute without re-capturing.

3.3 Composite scoring and intent resolution

An intent is embedded and matched against the graph; candidates are ranked by a composite of semantic similarity, observed reliability, freshness, and faithful replay. The chosen route is executed; a content-addressed cache stores each intermediate result under the hash of its content, so a second visit resolves without launching a browser. [shipped]

3.4 Skill lifecycle and schema drift

Routes decay — schemas drift, auth flows mutate. The lifecycle tracks reliability and demotes routes that stop reproducing, falling back to capture when a route goes stale. The full accountability machinery that makes a freshness claim costly to fake is the subject of the third paper [2] and is not relitigated here.

4 Economic model

4.1 The adoption condition

A cost-minimising agent will use the shared graph if and only if the fee to reuse a mapped route undercuts the cost of rediscovering it from a browser:

$$f_{\text{route}} \;<\; c_{\text{rediscovery}}. \tag{1}$$

The rediscovery cost decomposes into the browser's latency, compute, and token cost:

$$c_{\text{rediscovery}} \;=\; c_{\text{latency}} + c_{\text{compute}} + c_{\text{tokens}}. \tag{2}$$

Because the mapping cost is paid once and amortised over every later caller, (1) holds for any route reused more than a handful of times.

4.2 What agents pay for

Discovery and internal-API routing are free; only execution against a priced route settles. An *unpaid* `execute` *gets HTTP 402.* [shipped] Agents pay for the value they consume — a settled call against a maintained route — not for the act of looking.

4.3 Why a shared commons can emerge

Under (1) the dominant strategy for each agent is to reuse rather than re-map, and the dominant strategy for an indexer is to publish a route that others will reuse and earn on. The commons emerges because cooperation out-competes re-derivation for every party, not because participation is mandated.

5 Route-level payment architecture

All tiers are additive and collectively bounded by the adoption condition (1).

Tier 1: skill installation. A one-time fee to install a packaged skill, priced well below a single browser rediscovery.

Tier 2: opt-in per-execution. Per-call settlement on priced routes. Paid execution *settles over* ***x402***, and the settlement *splits three ways* — to the indexer who mapped the route, the platform, and (when claimed) the site owner — so *x402 routes USDC to all three* in one transaction. [shipped] This *three-way live split is shipped.* [shipped]

Tier 3: search and routing fees. An optional fee on a priced shortlist returned by resolution.

Dynamic route pricing. Prices track observed reliability and demand so a maintained, reliable route earns more than a stale one, keeping the incentive pointed at quality.

6 Quality proofing and validation

A reused route is only valuable if later agents can trust it. New submissions land in a shadow state until corroborated by independent reuse; reliability is scored from real outcomes (success ratio, consecutive failures, schema-drift signals); and a route that stops reproducing is demoted. The cryptographic accountability that makes a freshness claim *bondable* is developed in the companion papers [1, 2].

7 Implementation and evaluation

We evaluate the API-native path against a browser-automation baseline (Playwright) on a corpus of 94 live domains spanning search, listings, detail pages, and authenticated reads. Table 1 reports the headline results. Both paths make requests to the same target servers over the same network; the latency difference is browser execution overhead.

The reuse mechanism is isolated in a runnable reference benchmark: a warm cached replay categorically skips the cold mapping path, and the harness records a measured ≈95× mean speedup on the reuse micro-benchmark (`reference/bench/bench_reuse.py`), with a conservative ≈27×

Metric	Value
Domains evaluated	94
Mean latency (Unbrowse)	950 ms
Mean latency (Playwright)	3,404 ms
Mean speedup	3.6×
Median speedup	5.4×
Best single-domain speedup	30×
Mean cold-start (first capture)	12,400 ms
Tokens vs. browser baseline	40× fewer

Table 1: API-native vs. browser-automation baseline across 94 live domains.

observed gap on a single live URL (`reference/bench/live-measured-1url.json`). [reference] The full release-coverage methodology — corpus shape, scoring rubric, and current numbers — is maintained alongside the product.

Cost savings. The latency advantage is also a dollar advantage. A cold browser rediscovery of a route costs an estimated \$0.10–0.53 in compute and tokens (Table 2); the shared graph turns that into a one-time skill-installation fee of \$0.005–0.02, an order-of-magnitude reduction in the cost of obtaining a working route, after which reuse is a content-addressed lookup. The marketplace prices reuse below rediscovery by construction (Eq. 1), so the saving is structural, not promotional.

Action	Estimated cost
Browser rediscovery (cold, per route)	\$0.10 – \$0.53
Tier 1: skill installation (one-time)	\$0.005 – \$0.02
Reuse (content-addressed lookup)	≈\$0

Table 2: Per-route cost: browser rediscovery vs. shared-graph reuse.

Coverage. Speed and cost are worthless without coverage, so we measure whether the route graph actually returns real results across a broad, diverse intent space rather than on cherry-picked domains. A live coverage gate runs the real `unbrowse search` binary over a diverse intent set and counts the fraction that return real results; on the current graph it returns results on every probed intent (coverage = 1.0 against a ≥ 0.75 release threshold). [reference] Coverage is a gated, re-runnable measurement (`scripts/coverage-gate.sh`), not a static claim — it fails honestly when the graph cannot answer.

7.1 The per-call figure is a floor, not the headline

The 3.6× mean of Table 1 is a *per-call* comparison: one cached route execution against one cold browser rediscovery. A real agent task is not one call. It is a sequence of browser interactions — navigate, wait, click, wait, fill, submit, paginate — and each interaction costs latency, can fail and be retried, and re-reads a growing page into the agent's context, so token cost *compounds* across the heavy DOM reads. The API-native path collapses the same task into roughly one structured call: no multi-step navigation, no retry tax, no compounding context. Accounting for this honestly, *the per-call figure is a lower bound, not the end-to-end advantage*: the per-task

cost is the per-call cost multiplied by the task's click count, its failure-retry factor, and its token compounding. On a representative task (ten interactions, three DOM reads, an 80% per-step success rate) the model recovers ≈45× faster and ≈300× fewer tokens end-to-end, bracketing the field-measured ≈30× faster / ≈90× cheaper headline. We ship this as a deterministic, runnable reference (`reference/bench/task_cost.py`) so the compounding is provable arithmetic rather than a marketing multiplier. [reference]

8 Discussion

The first-party API is not always present (some surfaces are genuinely render-bound), and a learned route can drift. Unbrowse treats both honestly: the browser remains the bottom of a descent for the render-bound minority, and the lifecycle demotes drifting routes. What the wedge buys is the common case, which is most of the web: a structured request the site already makes, recovered once and shared. Two questions it deliberately leaves open — securing the descent below HTTP, and keeping the shared graph trustworthy as it scales — are the subjects of the two companion papers.

9 Conclusion

The graph is the asset. Every site an agent automates today is re-discovered from scratch, per agent, per call — the browser tax paid over and over. Turning that one-off rediscovery into a shared, reusable route graph removes the tax and makes maintenance, not discovery, the place value compounds. Internal APIs are most of what an agent needs to read and act on the web, and for that majority they are, in the affectionate sense of the title, all you need. They are also not the whole stack: securing the layers below HTTP is *Crypto Was All You Needed* [1], and keeping the graph fresh and bonded is the *Unbrowse Maintenance Network* [2]. This paper is the wedge they stand on.

A Reproducibility

The benchmark corpus, capture configuration, and scoring rubric are maintained with the product; the reuse mechanism is reproduced in isolation by `reference/bench/bench_reuse.py` ([reference]) so the categorical warm-vs-cold gap can be re-derived independently of any live endpoint.

B Benchmark domain list

The 94 evaluated domains span search engines, marketplaces, social and news feeds, developer platforms, and authenticated dashboards; the per-domain breakdown is distributed with the release-coverage methodology.

References

[1] L. Tham. *Crypto Was All You Needed: A Signed, Layer-Descending Stack for Agent Computation*. Unbrowse AI, 2026.

[2] L. Tham. *Unbrowse Maintenance Network: Proof of Indexing and Bonded Accountability in a Shared Route Graph*. Unbrowse AI, 2026.

[3] H. He et al. *WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models*. arXiv:2401.13919, 2024.

[4] X. Deng et al. *Mind2Web: Towards a Generalist Agent for the Web*. NeurIPS, 2023.

[5] S. Zhou et al. *WebArena: A Realistic Web Environment for Building Autonomous Agents*. ICLR, 2024.

[6] T. Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS, 2023.

[7] S. G. Patil et al. *Gorilla: Large Language Model Connected with Massive APIs*. arXiv:2305.15334, 2023.

[8] Anthropic. *Model Context Protocol*. 2024. https://modelcontextprotocol.io.

[9] Coinbase. *x402: An Open Protocol for Internet-Native Payments over HTTP 402*. x402.org whitepaper, 2025. https://github.com/coinbase/x402.

[10] E. Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.